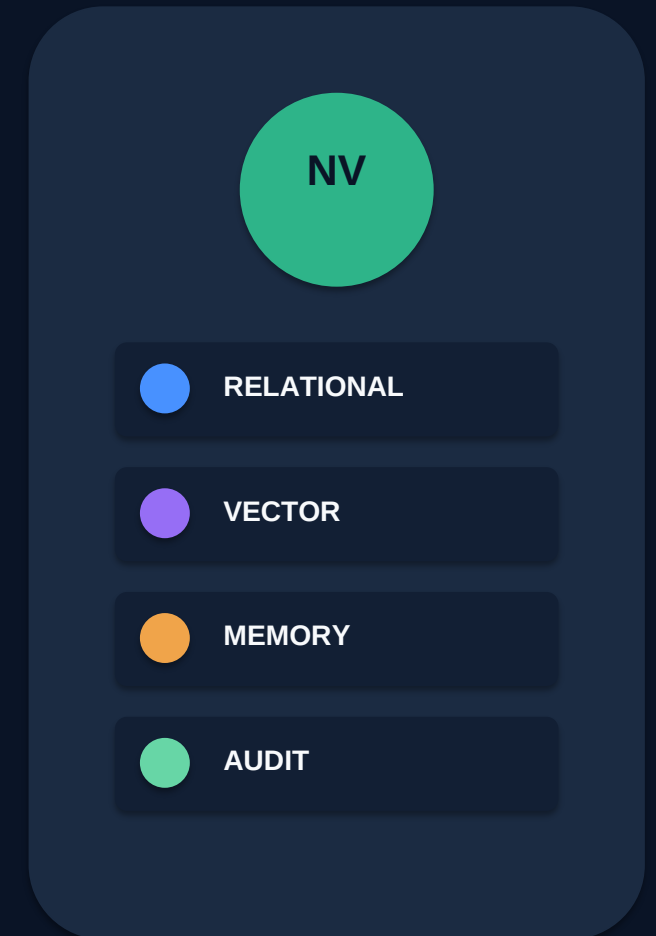


NeuroVault

MySQL-Native Cognitive Memory & Hybrid Relational RAG

A transactional database layer for persistent, explainable and evolving AI memory.



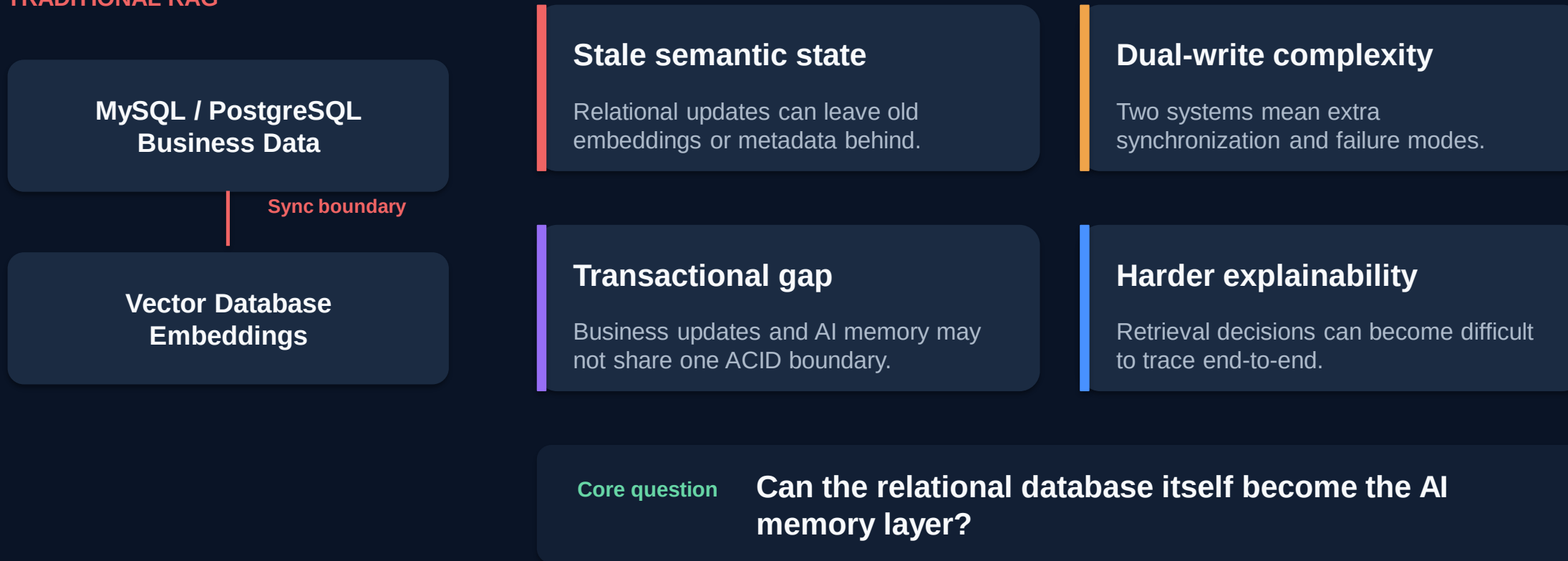
Sai Nikhit & Sohan

DBTHON'26 Innovation Challenge

AI memory is split across silos

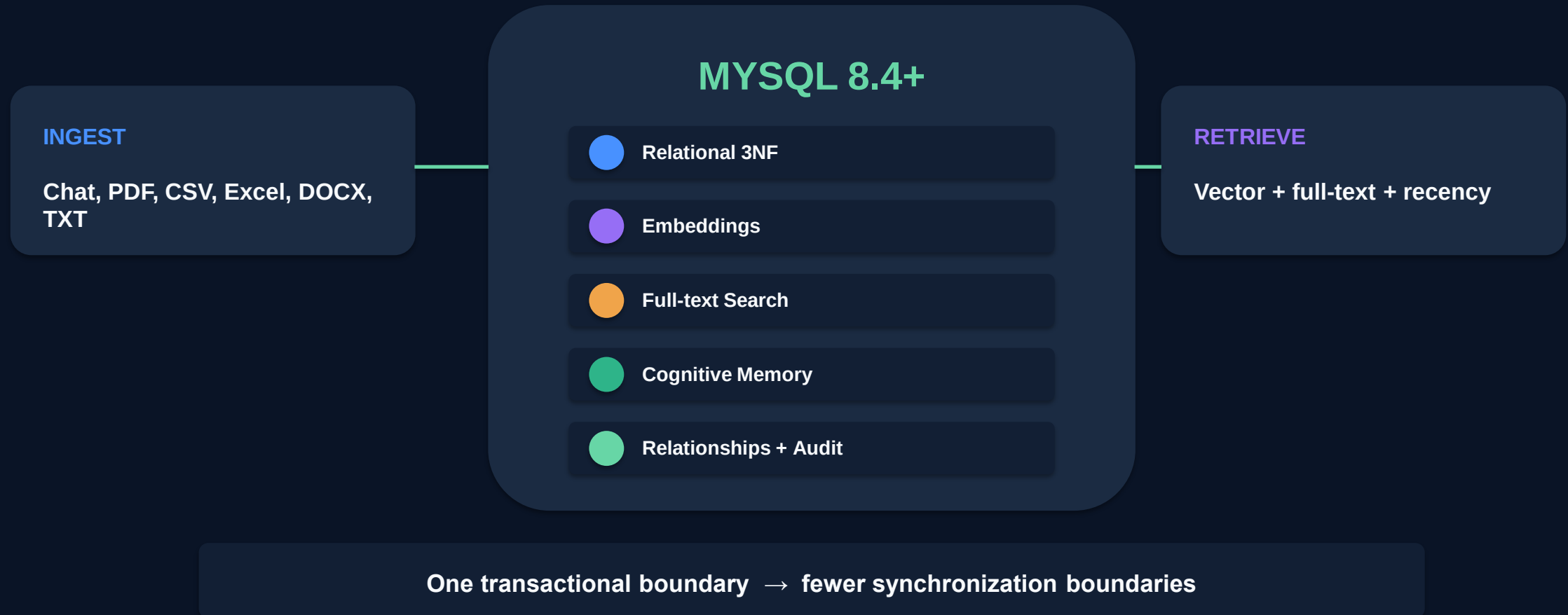
Traditional RAG often separates business truth from semantic memory.

TRADITIONAL RAG



Make MySQL the cognitive source of truth

NeuroVault co-locates relational, vector, memory, relationship and audit data.



What makes NeuroVault different

The project combines DBMS principles with persistent cognitive behavior.

01

Zero Vector Silos

Embeddings live alongside relational records inside the database.

02

Hybrid RAG

Semantic similarity + full-text relevance + temporal recency.

03

Ebbinghaus Memory

Repeated memories strengthen; unused memories decay.

04

Relational Graph

Multi-hop relationships using normalized MySQL structures.

05

Explainable Audit

Retrieval scores and system activity remain inspectable.

One query. Three signals. One ranked context.

NeuroVault makes retrieval mathematically inspectable.

“What payment method does the user prefer?”



VECTOR

Cosine similarity
Captures meaning & synonyms



FULL-TEXT

Keyword relevance
Finds explicit terms



RECENCY

Temporal signal
Prefers current memories

HYBRID SCORE → BEST CONTEXT

Example weighting in the dossier: 60% vector • 25% relational keyword rank • 15% recency

Memories do not all have equal lifetime

NeuroVault models memory strength using an Ebbinghaus-inspired decay mechanism.

RETENTION MODEL

$$R = \exp(-t / (S \cdot \log(\text{repetition} + 1)))$$

t = elapsed time • S = stability • repetition = recall frequency

MEMORY LIFECYCLE



Capture

New fact / event



Retrieve

Used in context



Reinforce

Stability increases



Decay

Unused memory fades

Goal: keep high-value knowledge fresh while reducing context bloat.

From files to verified database knowledge

The dossier describes PDF, CSV, Excel, DOCX and TXT ingestion.



Executive Brief

Plain-language explanation for reviewers.

Visual Charts

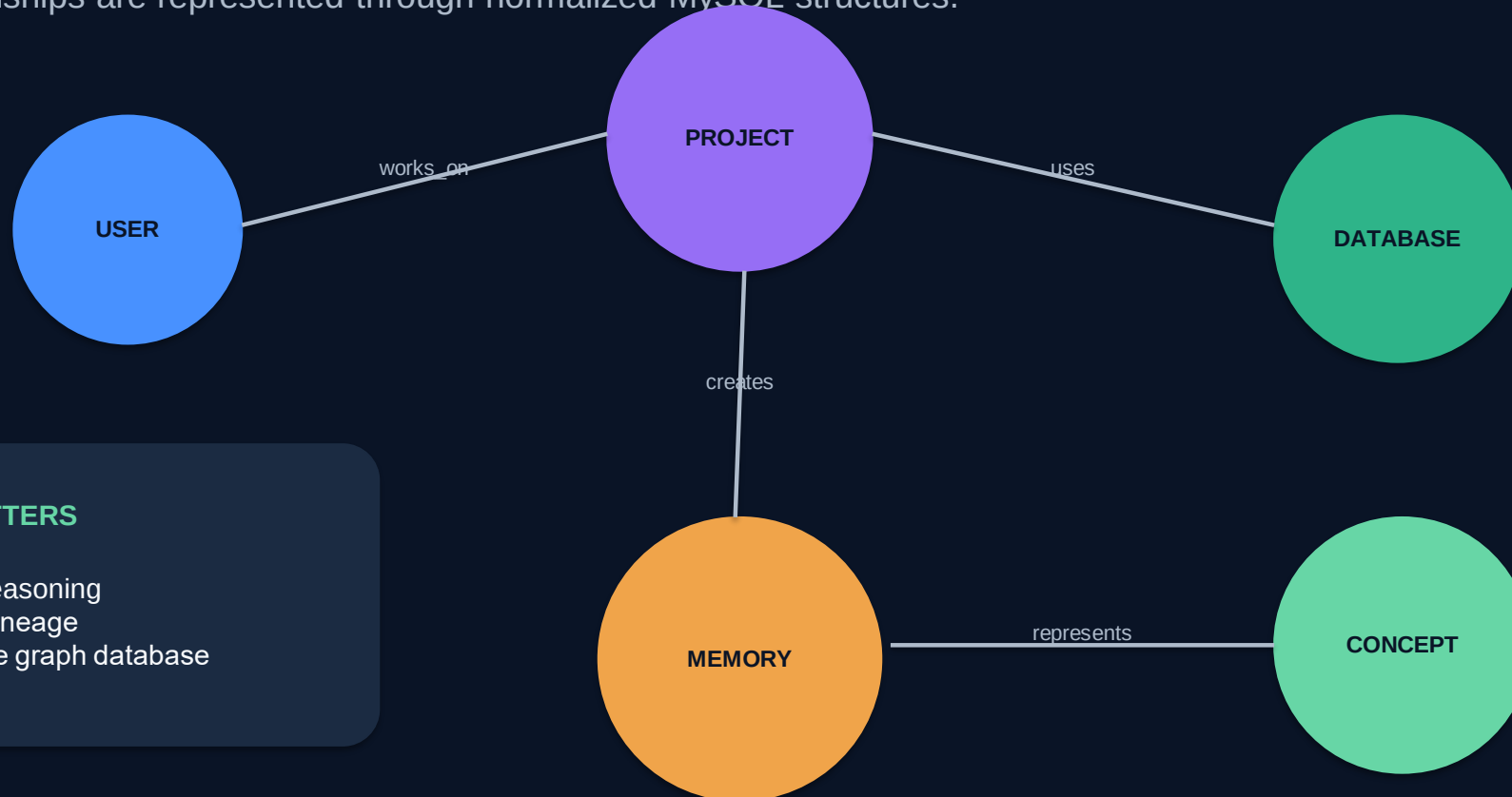
Recharts-style visual views of extracted data.

Empirical Facts + Raw Data

Inspect source-derived facts before committing.

Relational tables can become a cognitive graph

Graph relationships are represented through normalized MySQL structures.



WHY IT MATTERS

- Multi-hop reasoning
- Relational lineage
- No separate graph database required

Turn the RAG black box into an inspectable pipeline

Every retrieval can expose why a context item was selected.

RETRIEVAL EXPLANATION



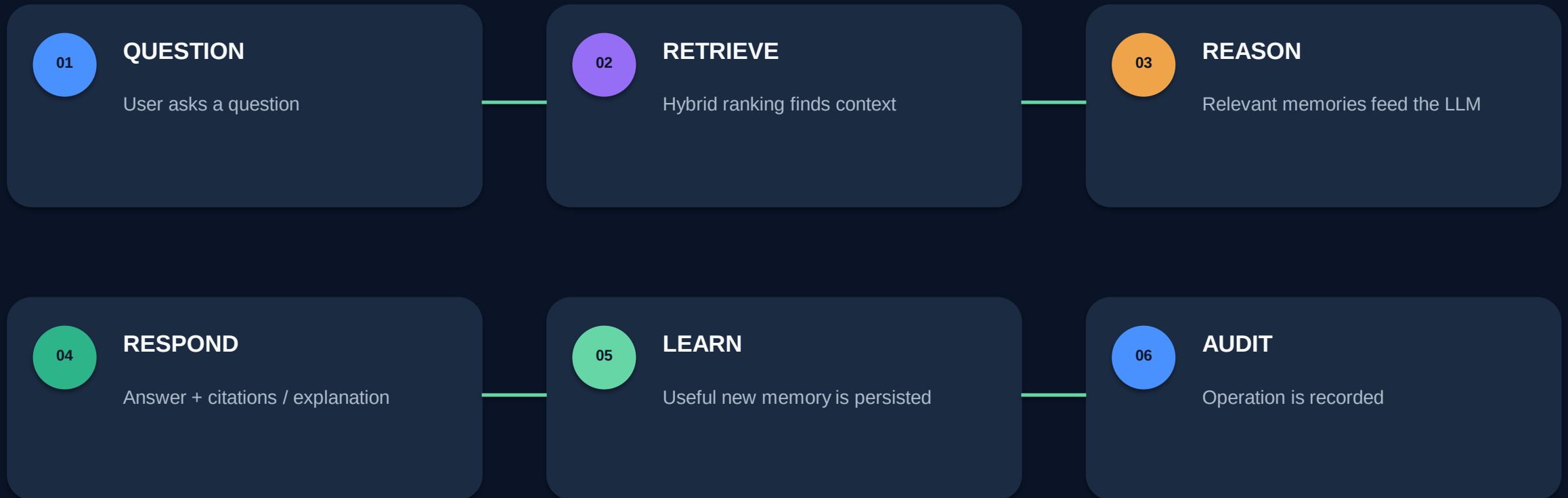
AUDIT TRAIL

10:42:08	MEMORY_RETRIEVED	12.4 ms
10:42:08	VECTOR_LOOKUP	4.1 ms
10:42:09	ANSWER_GENERATED	318 tokens
10:42:09	AUDIT_WRITTEN	SHA-256

Design goal: provenance, transparency and operational visibility.

How one user question moves through NeuroVault

The database is involved in ingestion, retrieval, memory and audit.



Performance claims presented in the reviewer dossier

Use these numbers only with the benchmark methodology available for your implementation.

Metric	Traditional Stack	NeuroVault	Reported Edge
Mean hybrid latency	48.6 ms	12.4 ms	74.5% faster
ACID commit isolation	No	Yes	Zero orphan vectors*
Retrieval recall	91.2%	96.8%	+5.6%
5k QPS P99	184 ms	24.8 ms	86.5% lower
Infrastructure cost	Higher	Standard MySQL	~65% savings

**“Zero orphan vectors” describes the architectural goal of shared transactional storage.*

Why NeuroVault matters

The project is not just another chatbot — it is a database architecture for AI memory.

“Turn the database from a passive data store into the cognitive memory layer of AI.”

DBMS Innovation

Relational + vector + memory + audit in one transactional boundary.

AI Innovation

Persistent memories evolve through retrieval, reinforcement and decay.

Reviewer Value

Explainable retrieval, graph relationships and measurable system telemetry.

Thank you

Sai Nikhit • Sohan | DBTHON 2026

NeuroVault — Entity Relationship Diagram

NeuroVault

Entity-Relationship Diagram (ERD)

MySQL-Native Cognitive Memory & Hybrid Relational RAG

Key Concepts Mapped

- Relational data (business truth) + Vector embeddings
- Cognitive memory with Ebbinghaus-like decay
- Relationships (graph-style, multi-hop)
- Hybrid retrieval (semantic + full-text + recency)
- Explainable audit (trace every action)

Sources & Documents

source		
PK	source_id	BIGINT
	source_type	ENUM (chat, pdf, csv, excel, docx, txt)
	source_name	VARCHAR(255)
	source_uri	VARCHAR(500)
	ingested_at	DATETIME
	metadata	JSON

document		
PK	doc_id	BIGINT
FK	source_id	BIGINT
	title	VARCHAR(255)
	content	TEXT
	content_type	VARCHAR(50)
	file_size	BIGINT
	ingested_at	DATETIME
	metadata	JSON

document_chunk		
PK	chunk_id	BIGINT
FK	doc_id	BIGINT
	chunk_index	INT
	content	TEXT
	token_count	INT
	metadata	JSON

Core Content (Business + Semantic)

entity		
PK	entity_id	BIGINT
	entity_type	VARCHAR(50) (user, product, policy, etc.)
	name	VARCHAR(255)
	description	TEXT
	attributes	JSON
	created_at	DATETIME
	updated_at	DATETIME

entity_document		
PK	entity_doc_id	BIGINT
FK	entity_id	BIGINT
FK	doc_id	BIGINT
	relevance_score	FLOAT
	created_at	DATETIME

embedding		
PK	embedding_id	BIGINT
FK	chunk_id	BIGINT
FK	doc_id	BIGINT
	moleance_score	FLOAT
	created_at	DATETIME

entity_relationship		
PK	rel_id	BIGINT
FK	source_entity_id	BIGINT
FK	target_entity_id	BIGINT
	relation_type	VARCHAR(50) (related_to, belongs_to, mentions, etc.)
	strength	FLOAT
	created_at	DATETIME

Memory (Cognitive Behavior)

memory		
PK	memory_id	BIGINT
FK	entity_id	BIGINT
FK	chunk_id	BIGINT
	memory_type	ENUM (fact, preference, event, etc.)
	content	TEXT
	strength	FLOAT (0.0 - 1.0)
	decay_factor	FLOAT
	last_accessed_at	DATETIME
	created_at	DATETIME
	metadata	JSON

memory_usage		
PK	usage_id	BIGINT
FK	memory_id	BIGINT
	used_at	DATETIME
	query_text	TEXT
	retrieval_score	FLOAT

Retrieval (Hybrid RAG)

retrieval_log		
PK	log_id	BIGINT
	query_text	TEXT
	top_k	INT
	filters	JSON
	created_at	DATETIME

retrieval_result		
PK	result_id	BIGINT
FK	log_id	BIGINT
FK	chunk_id	BIGINT
	score_vector	FLOAT
	score_fulltext	FLOAT
	score_recency	FLOAT
	final_score	FLOAT
	rank	INT

Audit (Explainability)

Audit_log		
PK	audit_id	BIGINT
	actor_type	ENUM (user, system, agent)
	actor_id	VARCHAR(100)
	action_type	VARCHAR(50) (ingest, retrieve, update, delete, etc.)
	table_name	VARCHAR(50)
	record_id	BIGINT
	details	JSON
	created_at	DATETIME

Legend

Primary Key (PK) FK Foreign Key (FK) — Relationship 1 One (cardinality) N Many (cardinality)